

Machine Learning in Practice I

Lecture 3

Antal Jakovac, 2025

Programming code structure

How to organize a (ML) code?

- points of view:
 - ➔ effective code development (also in teams)
 - ➔ scalability
 - ➔ extensibility
 - ➔ easy maintenance



Original approach: monolith program structure

- UI, business logic, data access in one code
- coding logic: *input data* → *transformation* → *resulting data*
- best fits for imperative/procedural languages (like C, Fortran)
 - ➔ program state: mutable variables
 - ➔ program is a chain of commands, changing the state
- natural approach → original programming style



problems:

- hard to trace bugs or change code:
 - variable change appear in different functions
 - frequently hidden state change in procedures
 - in a large code intractable structure
- functions are specific → not reusable
- experience: monolith codes become unmaintainable
(*software complexity crisis in the 1970s–80s*)
- still used for starting a project (MVP)

```
y = 3 #global state variable
```

```
def function(x):  
    x += y  
    y += 1 #changes y tacitly  
    return x
```



different type of solutions:

- object oriented programming
 - ➔ data and the corresponding procedures are bundled: "objects" of the world
 - ➔ abstraction: hierarchy of classes
 - ➔ inheritance: classes are reusable
 - ➔ examples: C++, Java

```
class MyClass:  
    def __init__(self, state):  
        self.state = state  
  
    #change of state is a class method  
    def rotate(self, angle):  
        self.state → self.state'(angle)  
        return self
```



- functional programming

- stateless programs, immutable variables
- functions give back new objects (pure functions)
- based on mathematical logic (λ -calculus)
e.g. $\lambda x. x+1$ and their application, e.g. $(\lambda x. x+1) 5 \rightarrow 6$
- basic elemental functions: see next page

```
counter=0
def impure():
    counter+=1 #side effect
    return counter
def pure(x):
    return x+1 #no side effect
```



- basic elemental functions

- map: `map ( $\lambda x. x*2$ ) [1,2,3]  $\rightarrow$  [2,4,6]`

- reduce/fold: `fold (+) 0 [1,2,3]  $\rightarrow$  6`

- filter: `filter even [1,2,3,4]  $\rightarrow$  [2,4]`

- zip: `zip ['a','b','c'] [1,2,3]  $\rightarrow$  [('a',1),('b',2),('c',3)]`

- compose: `(compose f g) x  $\rightarrow$  f(g(x))`

- examples: Haskell, Lisp, F#

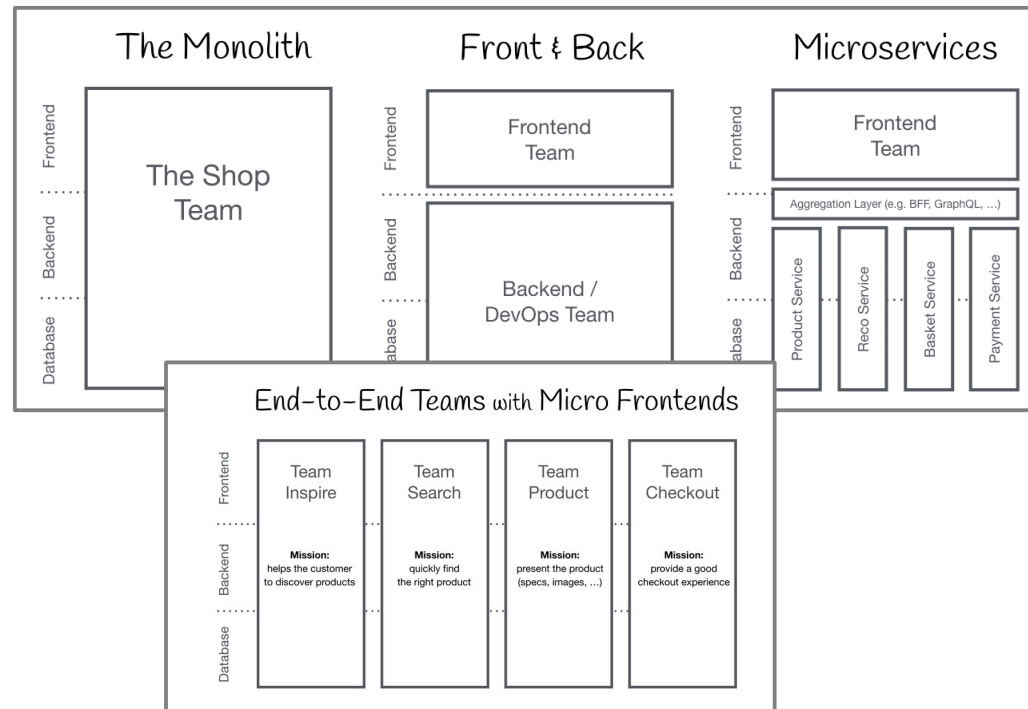
- not popular in the original form, but lot of elements are applied



- mixed (multi-paradigm) languages:
 - allow all kind of programming style
 - examples: Python, JavaScript, Rust, Swift,
 - e.g. Python has map, filter, reduce, based on lists.
 - Python supports OOP, classes, inheritance, etc.

Other way of deviating from monolithic imperative coding

- monolithic structure
- frontend+backend
- microservices
- micro frontend structure
(c.f. <https://micro-frontends.org/>)



Backend + frontend

- rise of web (1990s) browsers became universal clients
- specialized tools for browser programming (like HTML, CSS, JavaScript)
- task solving (backend) + visualization (frontend) separately
 - ➔ develop separately, different languages, tools
 - ➔ scale BE and FE separately
 - ➔ needs interface management → see next page



- functions designed for intra-code communication, can not be applied any more
- inter-service communication standards:
 - ➔ APIs (application programming interface → synchronous)
 - ➔ queues (message buffer → asynchronous)

- APIs – call server functions (endpoints)
 - ➔ needs continuously running background processes (web server process), listening to input channels (ports)
 - ➔ endpoints: well defined input and output formats, usually in json
 - ➔ communication → REST API: get, put/patch, post, delete
 - ➔ in Python: Flask, fastAPI
 - ➔ can be asynchronous, too

#fastAPI snippet

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Hello ✨"}
```

#start service

```
uvicorn main:app
```

#call service

```
open http://127.0.0.1:8000, or
curl http://127.0.0.1:8000/
results {"message": "Hello ✨"}
```



- queues: async send message
 - ➔ message broker runs in background, it stores messages
 - ➔ worker processes deliver messages for subscribers/consumers
 - ➔ data plain text or json
 - ➔ metadata + payload
 - ➔ in Python: queue + threading

#queue application example

```
import queue  
q = queue.Queue()
```

#put data to queue

```
q.put(item)
```

#read data from queue

```
item = q.get()
```

#... process data ...

```
q.task_done()
```



Microservices + frontend

- good scaling properties (e.g. web users) → requiring independence of elementary tasks became popular
- lot independent, narrowly scoped (micro) services
 - separate development (languages, tools, teams), deployment
 - separate scaling
 - resilience: bugs are confined, not affect the whole system
- examples Netflix, Amazon → hype in 2010s



- new challenge: how to organize a lot of services
- microservice management: docker, Kubernetes

#example Dockerfile

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY main.py .
```

```
RUN pip install fastapi uvicorn
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

#example docker-compose.yml

services:

service1:

build: ./service1

container_name: service1

ports:

- "8001:8000"

service2:

build: ./service2

container_name: service2

ports:

- "8002:8000"

docker-compose up --build



- new challenge: how to organize a lot of services
- CI/CD pipelines:
 - ➔ continuous improvement
 - ➔ continuous deployment
 - ➔ team communication
- example: github



- new challenge: how to organize a lot of services
- CI/CD pipelines:
 - ➔ continuous improvement
 - ➔ continuous deployment
 - ➔ team communication
- example: github

```
#example .github/workflows/ci.yml
name: CI
on:
  push: branches: ["main"]
  pull_request: branches: ["main"]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest
```



- new challenge: how to organize a lot of services

- CI/CD pipelines:

- continuous integration
- continuous deployment
- team communication

- example: github

#practical example: git clone project

```
git clone https://github.com/ajakovac/ML-in-Practice-I.git  
cd ML-in-Practice-I
```



- despite all advantages microservices have drawbacks
 - ➔ operational complexity → manage hundreds of services
 - ➔ network calls → latency, performance issues
 - ➔ interprocess debugging is hard



- Current trends (2020s → now)
 - ➔ modular monolith applications
 - ➔ microservices where they make sense
 - ➔ functions-as-a-service (FaaS)
 - in cloud services like AWS, Azure, Google cloud
 - locally they seem like functions
 - behave like services when deployed
 - run on demand



- Current trends

- ➔ modular n

- ➔ microserv

- ➔ functions-

- in clou

- locally

- behave

- run on

#example in AWS

#lambda_function.py

```
def lambda_handler(event, context):  
    name = event.get("name", "my_name")  
    return { "statusCode": 200,  
            "body": f"Hello, {name}! ✨"
```

zip function.zip lambda_function.py

aws lambda create-function \

--function-name helloLambda \

--runtime python3.12 \

--role arn:aws:iam::<YOUR_ACCOUNT_ID>:role/lambda-ex-role \

--handler lambda_function.lambda_handler \

--zip-file fileb://function.zip

curl -X POST \

-H "Content-Type: application/json" \

-d '{"name": "MLcourse"}' \

https://abcd1234.execute-api.us-east-1.amazonaws.com/default/hello

Code infrastructure → software architecture

- use few necessary microservices → clear task definition
- use lambda for simple services
- containerize microservices → portability, environmental stability
- use container management → docker (Kubernetes)
- use CI/CD management → github or similar

Data management sources, formats

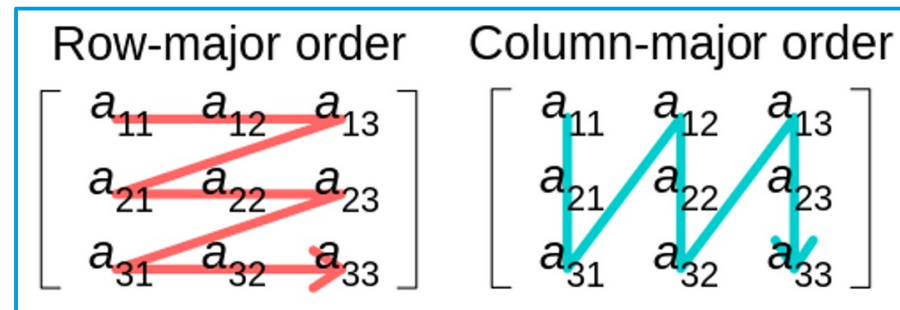


data sources:

- user input (text, image, video, etc.) – often not well formed, errors
- system generated (logs, measurements, predictions) – well formed data, easy-to-use
- own data, own clients (first-party), other companies, their own clients (second-party), other companies not own clients (third-party – e.g. internet usage, traffic, etc)

data formats

- CSV (comma separated values)
 - text, human readable
 - tabular, row-major
 - fast to get data belonging to the same example
- Parquet
 - binary – more compact, but not human readable
 - tabular, column major
 - fast to get data belonging to the same feature



data formats

- CSV (comma-separated values)

- text, human-readable
- tabular, row-oriented
- fast to get data out

- Parquet

- binary – more compact
- tabular, column-oriented
- fast to get data out

```
#example books.csv
```

```
title,author,publisher,year
```

```
The Hobbit,J.R.R. Tolkien,George Allen & Unwin,1937
```

```
The Fellowship of the Ring,J.R.R. Tolkien,George Allen & Unwin,1954
```

```
Nineteen Eighty-Four,George Orwell,Secker & Warburg,1949
```

```
Brave New World,Aldous Huxley,Chatto & Windus,1932
```

```
The Catcher in the Rye,J.D. Salinger,"Little, Brown and Company",1951
```

```
-----  
#read data using pandas
```

```
import pandas as pd
```

```
df = pd.read_csv("books.csv")
```

```
print(df)
```

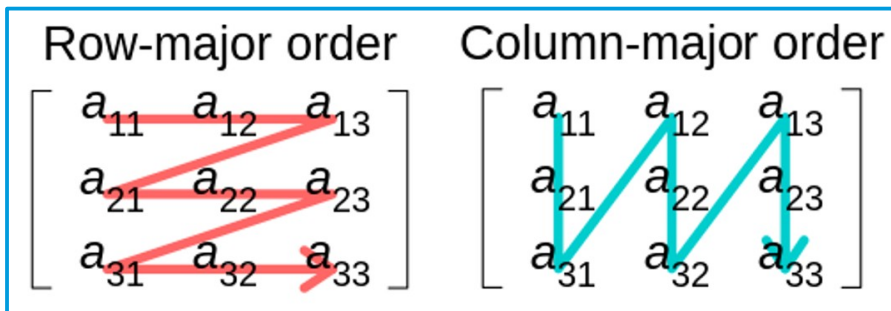
```
-----  
#output
```

	title	author	publisher	year
0	The Hobbit	J.R.R. Tolkien	George Allen & Unwin	1937
1	The Fellowship of the Ring	J.R.R. Tolkien	George Allen & Unwin	1954
2	Nineteen Eighty-Four	George Orwell	Secker & Warburg	1949
3	Brave New World	Aldous Huxley	Chatto & Windus	1932
4	The Catcher in the Rye	J.D. Salinger	Little, Brown and Company	1951

data formats

- CSV (comma separated values)

- text, human readable
- tabular, row-major
- fast to get data belonging to the same example



- Parquet

- binary – more compact
- tabular, column-major
- fast to get data belonging to the same column

```
#store in parquet – needs fastparquet to be installed  
df.to_parquet("books.parquet", engine="fastparquet", index=False)  
-----  
#store in parquet – needs fastparquet to be installed  
df_parquet = pd.read_parquet("books.parquet", engine="fastparquet")
```

data formats:

- JSON (JavaScript Object Notation) → today's standard
 - hierarchical structure
 - text, human readable
 - key-value pairs
 - in Python: dict
 - supported by all languages

data formats:

- JSON (JavaScript Object Notation)

- hierarchical
- text, human-readable
- key-value pairs
- in Python: `dict`
- supported by many languages

```
#example books.json
```

```
[  
  {  
    "title": "The Hobbit",  
    "author": "J.R.R. Tolkien",  
    "publisher": "George Allen & Unwin",  
    "year": 1937  
  },  
  {  
    "title": "The Fellowship of the Ring",  
    "author": "J.R.R. Tolkien",  
    "publisher": "George Allen & Unwin",  
    "year": 1954  
  },  
  ...  
]
```

```
#read with pandas
```

```
df = pd.read_json("books.json")
```

END OF LECTURE 3